



Facultad de Ciencias
Sociales y Administrativas

Licenciatura en Informática y
Desarrollo de Software

PRÁCTICO DE ENSEÑANZA

ASIGNATURA: Compiladores

PRÁCTICO N°: 1 **FECHA:** 14/08/26

“Análisis Léxico – Expresiones Regulares y Tokens”

DOCENTES RESPONSABLES:

Prof. Zufia Marcelo, Prof. Gonzalez Juan

NOMBRE Y APELLIDO DEL ALUMNO: Candela Puerta

CURSO Y COMISIÓN: 3A

OBJETIVO:

- Comprender los fundamentos del análisis léxico dentro del proceso de compilación.
- Comprender la diferencia entre **token** y **lexema**.
- Construir y utilizar **Expresiones Regulares** para reconocer patrones.
- Implementar un analizador léxico básico utilizando C, Java o Python.
- Reconocer y clasificar diferentes tipos de tokens.
- Detectar errores léxicos en una entrada.

I. Consignas

A. Investiga y explica brevemente:

■ ¿Qué es un compilador?

Un compilador es un programa que traduce código escrito en un lenguaje de programación de alto nivel, llamado código fuente, a otro lenguaje, generalmente código máquina o código intermedio, para que pueda ser ejecutado por una computadora.

■ ¿Qué es el análisis léxico?

El análisis léxico es la primera etapa del proceso de compilación. Consiste en leer el código fuente como una secuencia de caracteres y agruparlos en unidades con significado llamadas tokens.

■ ¿Qué función cumple el analizador léxico?

El analizador léxico se encarga de:

- Leer los caracteres del código fuente.
- Agruparlos para formar lexemas.
- Clasificar cada lexema dentro de un tipo de token.
- Ignorar elementos no significativos, como espacios, tabulaciones y comentarios.
- Detectar errores léxicos, como símbolos que no pertenecen al lenguaje.
- Entregar la secuencia de tokens al analizador sintáctico.

■ ¿Qué es un token?

Un token es una categoría o tipo de elemento reconocido en el código fuente.

■ ¿Qué es un lexema?

Un lexema es la secuencia concreta de caracteres del código fuente que coincide con el patrón de un token.

■ ¿Cuál es la diferencia entre token y lexema?

El lexema es el texto concreto encontrado en el código, mientras que el token es la categoría a la que pertenece ese texto.

- ¿Qué relación existe entre Expresiones Regulares y análisis léxico?

Las expresiones regulares permiten definir los patrones que deben cumplir los lexemas para ser reconocidos como determinados tokens. El analizador léxico utiliza estos patrones para clasificar los fragmentos del código fuente.

II. Práctica

A. Expresiones Regulares

- Identificador

Debe comenzar con una letra y posteriormente podrá contener:

- letras;
- números;
- _.

- Ejemplos válidos:

- nombre
- contador
- contador1
- usuario_1
- _variable

- Ejemplos inválidos:

- 123nombre
- 1usuario
- usuario-
- @nombre

- Indicar:

- Expresión Regular.

`^[a-zA-Z][a-zA-Z0-9_]*$`

- Ejemplos válidos.

Nombre

- Ejemplos inválidos.

123nombre

- Explicación de la expresión utilizada.

^ Inicio de la cadena

[a-zA-Z] Primera posición: letra o guion bajo

[a-zA-Z0-9_]* Posiciones siguientes: letras, números o guion bajo

\$ Final de la cadena

B. Número entero

- Reconocer números enteros.

Ejemplos:

- 10
- 125
- 999
- -15
- +20

- Indicar:

- Expresión Regular.
^[+-]?[0-9]+\$
- Ejemplos válidos.
10
- Ejemplos inválidos.
12.5
- Explicación de la expresión utilizada.
 ^ **Inicio de la cadena**
 [+]? **Signo positivo o negativo opcional**
 [0-9]+ **Uno o más dígitos entre 0 y 9**
 \$ **Final de la cadena**

C. Número decimal

- Reconocer números decimales.

Ejemplos:

- 10.5
- 3.14
- 25.00
- -15.75

- Indicar:

- Expresión Regular.
^[+-]?[0-9]+\.[0-9]+\$
- Ejemplos válidos.
15.4
- Ejemplos inválidos.
abc
- Explicación de la expresión utilizada.
 ^ **Inicio de la cadena**
 [+]? **Signo positivo o negativo opcional**
 [0-9]+ **Uno o más dígitos antes del punto**
 \. **Punto decimal obligatorio**
 [0-9]+ **Uno o más dígitos después del punto**
 \$ **Final de la cadena**

D. Cadena de caracteres

- Reconocer cadenas encerradas entre comillas dobles.

Ejemplos:

- "Hola"
- "Juan"
- "Compiladores"
- "Hola mundo"

- Indicar:

- Expresión Regular.
^[a-zA-Z]+\$
- Ejemplos válidos.
"hola"
- Ejemplos inválidos.
123
- Explicación de la expresión utilizada.
 ^ **Inicio de la cadena**
 " **Comilla doble de apertura**
 [a-zA-Z]+ **Una o más letras o espacios**

"	Comilla doble de cierre
\$	Final de la cadena

III. Reconocimiento de palabras reservadas

A. Construye una expresión que permita reconocer las siguientes palabras reservadas:

- if
- then
- else
- while
- for
- def
- return
- int
- float
- string
- bool

El programa deberá diferenciar:

- if

De:

- if123

La primera deberá reconocerse como:

- PALABRA_RESERVADA

Mientras que la segunda deberá reconocerse como:

- IDENTIFICADOR

$\wedge(\text{if}|\text{then}|\text{else}|\text{while}|\text{for}|\text{def}|\text{return}|\text{int}|\text{float}|\text{string}|\text{bool})\$$

IV. Reconocimiento de operadores

Construye expresiones regulares para reconocer:

A. Operadores aritméticos

- +
 - -
 - *
 - /
 - %
- $\wedge[+\-*/\%]\$$**

B. Operadores relacionales

- >
- <
- >=
- <=
- ==

- !=

^(>|=|<|=|!=|>|<)\$

C. Operador de asignación

- =

^=\$

D. El programa deberá diferenciar correctamente:

- =
- ==
- >=
- <=

^(>|=|<|=|!=|>|<|=)\$

V. Reconocimiento de delimitadores

A. El analizador deberá reconocer:

- (
-)
- [
-]
- {
- }
- ;
- ,
- .

B. Para cada símbolo deberá indicar:

- TOKEN
- LEXEMA

Por ejemplo:

- DELIMITADOR ;

VI. Implementación del primer analizador léxico

A. Desarrolla un programa en C, Java o Python que permita ingresar una cadena por teclado y determine los tokens presentes.

Por ejemplo, para:

- A = 10;

B. deberá producir:

TOKEN	LEXEMA
IDENTIFICADOR	A

ASIGNACION	=
NUMERO	10
DELIMITADOR	;

El script deberá:

- recibir una cadena;
- recorrerla;
- separar lexemas;
- asignar tokens;
- mostrar el resultado.

VII. Analizador de expresiones

A. Reconocer una expresión que combina distintos tipos de tokens.

B. El analizador deberá reconocer:

- $A = 10 + B;$

C. La salida esperada será:

IDENTIFICADOR	A
ASIGNACION	=
NUMERO	10
OPERADOR_ARITMETICO	+
IDENTIFICADOR	B
DELIMITADOR	;

Importante: El programa solamente deberá reconocer los tokens.

No deberá determinar todavía si la expresión es sintácticamente correcta.

VIII. Análisis de expresiones relacionales

A. El analizador deberá reconocer:

- $edad \geq 18;$

B. El resultado esperado será:

- IDENTIFICADOR
- OPERADOR_RELACIONAL
- NUMERO
- DELIMITADOR

C. Es decir:

IDENTIFICADOR	edad
OPERADOR_RELACIONAL	>=
NUMERO	18
DELIMITADOR	;

D. Casos de prueba

- edad > 18;
- edad >= 18;
- edad < 18;
- edad <= 18;
- edad == 18;
- edad != 18;

IX. Análisis de cadenas

A. El analizador deberá reconocer:

- nombre = "Juan";

B. Reconocer:

- IDENTIFICADOR
- ASIGNACION
- CADENA
- DELIMITADOR

C. El resultado deberá ser:

IDENTIFICADOR	nombre
ASIGNACION	=
CADENA	"Juan"
DELIMITADOR	;

X. Análisis de comentarios

A. El analizador deberá reconocer comentarios de una línea:

- // comentario
- // Hola mundo
- edad = 18; // edad

B. El programa deberá identificarlo como:

- COMENTARIO

Opcionalmente, deberá permitir descartarlo de la secuencia final de tokens.

XI. Manejo de espacios

A. Determinar cómo debe comportarse el analizador frente a espacios, tabulaciones y saltos de línea:

B. Por lo tanto:

- $A=10+B;$

con:

- $A = 10 + B;$

y:

- $A = 10 + B;$

Deberán producir la misma secuencia de tokens.

XII. Manejo de errores léxicos

El analizador deberá detectar caracteres que no formen parte del lenguaje definido, es decir, detectar caracteres que no pertenecen al lenguaje definido.

Por ejemplo:

- $A = 10 @ B;$

En el ejemplo, el carácter “@” no pertenece a ninguno de los tokens definidos.

A. Deberá generar:

IDENTIFICADOR	A
ASIGNACION	=
NUMERO	10

ERROR LEXICO

Caracter no reconocido	@
ASIGNACION	B
DELIMITADOR	;

Además, deberá informar:

- carácter problemático;
- línea;
- columna.

Casos inválidos:

- $A = 10 @ B;$
- $\text{precio} = 25 \# \text{cantidad};$

- usuario = \$100;

Importante

*El analizador no deberá detenerse ante el primer error.
Deberá informar el error y continuar procesando.*

XIII. Prioridad de reconocimiento

El analizador deberá analizar qué sucede con la ambigüedad producida por operadores que comparten prefijos:

- =
- ==
- >
- >=
- <
- <=

A. Explicar por qué el analizador debe reconocer:

- ==

como un único token y no como:

- =
- =

Implementar una solución que permita resolver este problema.

El analizador deberá seleccionar la coincidencia válida más larga.

Casos de prueba

- A = B;
- A == B;
- A > B;
- A >= B;
- A < B;
- A <= B;
- A != B;

Resultado esperado

Por ejemplo:

- A >= B;

Debe generar:

- IDENTIFICADOR
- OPERADOR_RELACIONAL
- IDENTIFICADOR
- DELIMITADOR

XIV. Línea y columna

A. Realizar un analizador para indicar la posición de cada token:

Cada token deberá almacenar:

- TIPO
- LEXEMA
- LINEA
- COLUMNA

B. Por ejemplo:

- `int contador = 10;`
- `float precio = 25.50;`

LINEA	COLUMNA	TOKEN	LEXEMA
1	1	PALABRA_RESERVADA	int
1	5	IDENTIFICADOR	contador
1	...	ASIGNACION	=

C. Pruebas

Utilizar código con varias líneas y espacios:

```
int edad = 25;
```

```
float altura = 1.83;
```

```
edad = edad + 1;
```

XV. Analizar un archivo

A. Crear un analizador que permita procesar un archivo de código fuente.

Por ejemplo:

- programa.txt (script a elección)

Con (script de ejemplo):

```
int contador = 10;
```

```
float precio = 25.50;
```

```
contador = contador + 1;
```

B. El programa deberá:

- abrir programa.txt;
- leer el contenido;
- analizarlo;
- reconocer los tokens;
- detectar errores;
- indicar línea y columna.

C. Caso adicional

Agregar al archivo:

- @

Comprobar que el analizador informe el error sin detener completamente el análisis.

XVI. Generación de tabla de tokens

Objetivo

Representar formalmente el resultado del análisis léxico.

A. Entrada de ejemplo:

int contador = 10;

float precio = 25.50;

contador = contador + 1;

B. Para cada archivo analizado deberá generarse una tabla con:

- PALABRA_RESERVADA
- IDENTIFICADOR
- ASIGNACION
- NUMERO
- DELIMITADOR

C. Para la entrada de ejemplo la tabla resultante sera:

Linea	Columna	Token	Lexema
1	1	PALABRA_RESERVADA	int
1	...	IDENTIFICADOR	contador
1	...	ASIGNACION	=
1	...	NUMERO	10
1	...	DELIMITADOR	;

XVII. Caso de prueba integral

A. Integrar todos los patrones y reglas desarrollados durante el TP en un único analizador, cuyo objetivo es comprobar que el analizador léxico sea capaz de procesar un fragmento de código fuente que contenga diferentes tipos de tokens.

B. Código fuente a analizar

int edad = 25;

float altura = 1.83;

```
if edad >= 18 {  
    edad = edad + 1;  
}
```

// Fin del programa

C. Ejecutar el analizador léxico sobre el código anterior.

D. El programa deberá identificar todos los tokens presentes, indicando como mínimo:

- tipo de token;
- lexema;
- línea;
- columna.

No deberá analizar si la estructura del if es sintácticamente correcta.

E. Resultado esperado

La secuencia deberá contener, entre otros, los siguientes tokens:

Lexema	Token
int	PALABRA_RESERVADA
edad	IDENTIFICADOR
=	ASIGNACION
25	NUMERO_ENTERO

Importante: El analizador deberá limitarse al análisis léxico.

El comentario:

- *// Fin del programa*

Deberá ser reconocido según la decisión tomada en el analizador de comentarios

F. Se deberá:

- Ejecutar el analizador.
- Obtener la secuencia completa de tokens.
- Presentar la tabla de tokens.
- Verificar línea y columna de cada token.
- Explicar cómo fueron reconocidos los operadores.
- Explicar cómo fueron reconocidos los números enteros y decimales.
- Indicar cómo se trató el comentario.
- Indicar la cantidad total de tokens reconocidos.

G. Entrega

- Código utilizado;
- Entrada utilizada;
- Tabla de tokens;
- Salida obtenida;
- Explicación del resultado.

XVIII. Casos inválidos

A. Verificar que el analizador pueda detectar entradas que no cumplen con los patrones léxicos definidos.

B. Probar el programa utilizando:

- 123abc
- @usuario
- "Hola
- 12.5.8
- A === B
- #contador

C. Para cada caso deberá indicar:

- Entrada:
- Tipo de error:
- Lexema/caracter problemático:
- Línea:
- Columna:

Ejemplo

Para:

- @usuario

La salida podría ser:

Entrada	@usuario
Tipo de error	ERROR LEXICO
Lexema/caracter problemático	@
Línea	1
Columna	1
Acción	informar error y continuar el análisis

Importante: El objetivo de este ejercicio es evaluar errores léxicos, no errores sintácticos.

Por ejemplo:

- A === B

Deberá analizarse desde el punto de vista de los tokens definidos.

D. Requisito de recuperación

Siempre que sea posible, el analizador deberá:

- detectar el error;
- informar su ubicación;

- informar el carácter o lexema problemático;
- continuar procesando el resto de la entrada.

XIX. Integración final

A. Desarrollar una versión final del analizador del *ejercicio XVII* que permita:

- *Ingresar una cadena.*
- *Analizar un archivo.*
- *Generar tokens.*
- *Informar lexemas.*
- *Informar línea y columna.*
- *Detectar errores léxicos.*
- *Mostrar estadísticas del análisis.*

B. una interfaz mínima

```
=====
ANALIZADOR LÉXICO
=====
```

1. *Ingresar cadena*
2. *Analizar archivo*
3. *Mostrar tokens*
4. *Mostrar errores*
5. *Mostrar estadísticas*
6. *Salir*

Seleccione una opción:

C. Si se analiza:

```
int edad = 25;
float altura = 1.83;

edad = edad + 1;
```

El programa podría mostrar:

```
=====
RESULTADO DEL ANÁLISIS
=====

Tokens reconocidos: 17
Errores léxicos: 0
Líneas analizadas: 4

=====
```

D. Luego podrá mostrar la tabla correspondiente

E. En el caso de que contenga errores:

```
int 123abc = 10;
edad @ 18;
precio = 12.5.8;
```

F. La salida deberá informar los errores encontrados, por ejemplo:

=====

ERRORES LÉXICOS

=====

Error 1
Lexema: 123abc
Línea: 1
Columna: 5

Error 2
Lexema: @
Línea: 2
Columna: 6

Error 3
Lexema: 12.5.8
Línea: 3
Columna: 10

G. Restricción importante:

El programa NO deberá realizar análisis sintáctico.

H. No deberá construir:

- árboles sintácticos;
- árboles de derivación;
- AST;
- tablas de parsing;
- análisis de gramática.